Paradigmas de Programación

Proyecto 1: Programación Orientada a Objetos en C++

Aprendizaje Esperado:

Construir programas basados en el paradigma orientado a objetos para resolver problemas que requieren modelar contextos complejos.

Problema a resolver:

Programar un sistema de gestión de rutinas de entrenamiento personalizadas.

Objetivo:

Desarrollar una aplicación en C++, aplicando los principios de la programación orientada a objetos, que permita gestionar ejercicios y rutinas de entrenamiento.

Contexto del problema

En un centro de entrenamiento deportivo, los instructores deben diseñar rutinas personalizadas para sus clientes según sus objetivos físicos.

Cada rutina está compuesta por distintos ejercicios, los cuales pueden variar en su forma de ejecución, duración e impacto en el cuerpo.

En el centro se trabajan principalmente dos tipos de ejercicios:

- Ejercicios cardiovasculares
- Ejercicios de fuerza

Si bien todos los ejercicios comparten ciertas características generales, cada tipo presenta particularidades que deben ser consideradas al momento de modelar el sistema.

Características de los ejercicios:

Todo ejercicio en el sistema debe considerar, al menos, la siguiente información:

- Código identificador (único).
- Nombre del ejercicio.
- Nivel de intensidad (Básico, Intermedio, Avanzado, Alto rendimiento).
- Tiempo estimado en minutos.
- Descripción detallada de su ejecución.

Particularidades según tipo de ejercicio:

Ejercicios cardiovasculares: Se caracterizan por enfocarse en la resistencia y el trabajo continuo. Además de la información general, deben registrarse aspectos como: distancia a recorrer y velocidad promedio sugerida.

El gasto energético en este tipo de ejercicios suele depender principalmente del tiempo de ejecución y la intensidad del esfuerzo. Para calcular el gasto energético de este tipo de ejercicio se puede utilizar cualquiera de las siguientes fórmulas:

gasto_energético = gasto_base + (distancia * 10)

o

gasto_energético = gasto_base * (1 + velocidad / 10)

donde

gasto_base = tiempo * factor_intensidad

donde el factor intensidad de obtiene de la siguiente tabla:

Nivel de intensidad	Factor
Básico	5
Intermedio	8
Avanzado	12
Alto rendimiento	15

Ejercicios de fuerza: Se enfocan en el desarrollo muscular mediante esfuerzos repetitivos. Además de la información general, deben considerar: número de repeticiones, número de series y peso utilizado. En este caso, el gasto energético está más relacionado con la cantidad de repeticiones, series y la carga utilizada. Para calcular el gasto energético de este tipo de ejercicio se utiliza la siguiente fórmula:

gasto_energético = gasto_base + (series * repeticiones * peso * 0.1)

donde

gasto_base = tiempo * factor_intensidad

donde el factor intensidad de obtiene de la siguiente tabla:

Nivel de intensidad	Factor
Básico	5
Intermedio	8
Avanzado	12
Alto rendimiento	15

Gestión de rutinas de entrenamiento:

El sistema debe permitir generar rutinas de entrenamiento solicitando la cantidad de ejercicios cardiovasculares que contendrá la rutina junto con el nivel de intensidad deseado de cada ejercicio cardiovascular. También, debe solicitar la cantidad de ejercicios de fuerza junto con el nivel de intensidad deseado de cada ejercicio de fuerza. Además, el sistema debe poder mostrar los nombres de los ejercicios que componen la rutina, agrupados en cardiovasculares y de fuerza, y poder calcular el tiempo total estimado de la rutina.

Resumen de las funcionalidades del sistema

El sistema debe permitir:

- Gestión de ejercicios:
    - Registrar nuevos ejercicios.
    - Eliminar ejercicios existentes.
    - Consultar la información de un ejercicio incluyendo su gasto energético.
    - Listar ejercicios según nivel de intensidad.
- Gestión de rutinas de entrenamiento:
    - Crear rutinas.
    - Consultar la información de una rutina incluyendo su tiempo estimado de duración.

Interfaz:

El programa debe ejecutarse en consola mediante un menú interactivo.

Requerimientos de diseño:

- Proponer un diseño orientado a objetos que modele adecuadamente el problema.
- Elaborar un diagrama de clases.
- Justificar las decisiones de diseño adoptadas.

Se espera que la solución considere:

- Abstracción.
- Encapsulamiento.
- Herencia.
- Polimorfismo.

Se recomienda proponer una organización de códigos donde las implementaciones de las clases se realicen en archivos .h (header files) y la función principal se defina en un archivo .cpp.

Entregables:

Informe en documento PDF contentivo de:

- Portada: identificación de la universidad, facultad, carrera y asignatura. Título del trabajo. Nombre y RUT de cada integrante del equipo, NRC de la sección, ciudad y fecha de entrega del informe.
- Introducción: motivación y objetivo del trabajo.
- Diseño de la solución: Diagrama de Clases donde se observen las relaciones entre las mismas.
- Descripción de la solución: explicación del modelo propuesto, clases diseñadas, relaciones entre ellas y justificación de decisiones.
- Conclusión: reflexión sobre los desafíos superados y los aprendizajes obtenidos.
- Tanto el informe como los archivos del sistema deberán comprimirse en una carpeta y publicarse en la sección Tareas del aula Canvas.

Software:

- Archivo (o archivos) con los códigos fuente del sistema.
- Sus códigos deben estar extensamente comentados, es decir, debe utilizar comentarios para describir las clases, los atributos y las funcionalidades de los métodos, además de clarificar la lógica de la programación de los mismos.
- El comentario inicial debe contener: nombre de la asignatura, NRC de la sección, breve descripción de las funcionalidades del programa y los datos de sus autores (nombres completos y RUT respectivos).
- Sólo puede utilizar bibliotecas estándar (que no requieran instalación adicional).
- Tanto el informe como los archivos del sistema deberán comprimirse en una carpeta y publicarse en la sección Tareas del aula Canvas.

Exposición:

- El grupo debe preparar una exposición, que no debe exceder los 15 minutos, en la cual cubran los siguientes aspectos:
    - Presentación del grupo.
    - Explicación del diseño de la aplicación (Diagrama de Clases).
    - Demostración de la ejecución de la aplicación.
    - Explicación general de la codificación de la aplicación (cada integrante del grupo explicará brevemente algún aspecto de programación del proyecto).
    - Conclusiones: dificultades superadas y aprendizajes adquiridos.

Requerimientos adicionales:

- Utilizando el paradigma de Programación Orientada a Objetos, se debe elaborar un programa en C++ que utilice arreglos para el almacenamiento de los datos.
- Se deben programar los métodos que sean necesarios para cubrir todas las funcionalidades requeridas, además de los métodos constructores, setters, getters y visualizadores.
- La actividad debe ser desarrollada por **todos** los integrantes del grupo de trabajo.
- Se recomienda analizar la rúbrica de esta actividad de evaluación y aclarar cualquier duda antes de iniciar la elaboración de su tarea.